

Reward Modeling for Multi-Agent Orchestration

Andrew Young
Automate Capture Research

June 13, 2026

Abstract

We present `orchestrator_rm`, an installable Python package that implements a self-supervised Bradley–Terry reward model for evaluating the quality of multi-agent orchestration traces. The package provides a complete pipeline: data collection, pairwise preference generation, model training, and inference, together with a lightweight orchestration framework that can be used to generate candidate traces. All components are organized into a clear module hierarchy, and a comprehensive test suite validates the end-to-end flow, the numerical stability of the Bradley–Terry loss, and the monotonicity of the learned reward with respect to engineered trace features. No empirical performance numbers are reported; instead we describe the methodology and the qualitative guarantees exercised by the tests. The implementation serves as a reproducible research baseline for future work on reward modeling in multi-agent systems.

1 Introduction

Coordinating multiple autonomous agents to achieve a common objective is a central challenge in robotics, distributed systems, and cloud orchestration. Traditional approaches rely on hand-crafted heuristics or rule-based schedulers, which often fail to capture subtle trade-offs such as latency versus resource consumption. Recent advances in reinforcement learning and preference-based learning suggest that a learned reward function can encode these trade-offs more flexibly. However, obtaining reliable supervision for a reward model is difficult: explicit reward signals are rarely available, and human annotation of full execution traces is costly.

We propose a self-supervised approach based on the Bradley–Terry (BT) model [1], which treats each pair of execution traces as a binary comparison. By generating synthetic preferences from engineered trace metrics (e.g., total execution time, number of context switches), we can train a neural reward model without external labels. The resulting model can be used to rank new orchestration candidates, guiding search or policy optimisation.

The contribution of this work is two-fold: (i) a fully packaged, pip-installable Python library (`orchestrator_rm`) that implements the BT reward model together with the surrounding data-generation and evaluation infrastructure; and (ii) a test suite that verifies the logical correctness of each component and the overall learning loop. The package is intended as a reference implementation for the research community.

2 Related Work

Preference-based reward learning has been explored in several domains. Christiano et al. [2] introduced deep RL from human preferences, while [3] applied pairwise ranking to robotic manipulation. The Bradley–Terry model, originally proposed for paired comparison data, has been adapted for learning to rank in information retrieval [4] and for preference-based RL [5].

Multi-agent orchestration research includes rule-based schedulers ? and learning-based controllers ?. Our work bridges these strands by providing a concrete, open-source implementation of a BT-based reward model specifically for orchestration quality.

3 Methodology

3.1 Problem Formulation

Let $\mathcal{T} = \{\tau_1, \dots, \tau_N\}$ denote a set of execution traces generated by a multi-agent orchestrator. Each trace τ is represented by a feature vector $\mathbf{x}_\tau \in \mathbb{R}^d$ (e.g., total runtime, memory usage, number of synchronization events). The goal is to learn a scalar reward function $r_\theta(\mathbf{x})$ parameterised by θ such that for any pair (τ_i, τ_j) the probability that τ_i is preferred over τ_j follows the Bradley–Terry formulation:

$$P(i \succ j \mid \theta) = \frac{\exp(r_\theta(\mathbf{x}_i))}{\exp(r_\theta(\mathbf{x}_i)) + \exp(r_\theta(\mathbf{x}_j))}.$$

3.2 Self-Supervised Preference Generation

Because explicit preferences are unavailable, we construct synthetic pairwise labels using a deterministic comparator $c(\tau_i, \tau_j)$ that encodes domain knowledge (e.g., lower runtime is better). For each generated pair we assign a binary label $y_{ij} = 1$ if $c(\tau_i, \tau_j) = \text{True}$ and $y_{ij} = 0$ otherwise. This yields a dataset $\mathcal{D} = \{(\mathbf{x}_i, \mathbf{x}_j, y_{ij})\}$ suitable for BT training.

3.3 Model Architecture

The reward model r_θ is a shallow feed-forward network:

$$r_\theta(\mathbf{x}) = \mathbf{w}^\top \sigma(\mathbf{W}\mathbf{x} + \mathbf{b}) + b_0,$$

where σ is a ReLU activation. The architecture is deliberately lightweight to keep training stable on modest hardware.

3.4 Training Objective

We minimise the negative log-likelihood of the BT model over $\mathcal{D}$:

$$\mathcal{L}(\theta) = - \sum_{(i,j) \in \mathcal{D}} [y_{ij} \log P(i \succ j \mid \theta) + (1 - y_{ij}) \log(1 - P(i \succ j \mid \theta))].$$

Gradient descent with Adam ? is used to optimise θ .

4 Implementation

The package follows the namespace `orchestrator_rm.<module>`. Table 1 summarises the core modules.

4.1 Testing Strategy

The test suite (located in `tests/`) contains unit and integration tests that verify:

- **Metric consistency:** `cost_metric` returns deterministic values for identical traces.
- **Pair generation:** For any two traces, the generated label matches the comparator c .

Module	Responsibility
<code>cost_metric</code>	Implements engineered trace metrics and the comparator $c(\cdot, \cdot)$.
<code>data_utils</code>	Utilities for serialising/deserialising trace feature vectors.
<code>encoder</code>	Optional embedding layer for high-dimensional raw trace data.
<code>pair_generator</code>	Generates pairwise training examples from a pool of traces.
<code>reward_model</code>	Definition of the feed-forward network and BT loss.
<code>orchestrator</code>	Minimal orchestrator that produces synthetic traces for testing.
<code>trace_collector</code>	Collects trace features during execution of the orchestrator.
<code>eval</code>	Inference utilities to rank new traces using the learned reward.

Table 1: Core modules of `orchestrator_rm`.

- **Loss stability:** The BT loss is finite and gradient-computable for random inputs.
- **Monotonicity:** After a single training step, the reward of a trace that is preferred by c is higher than its counterpart (within numerical tolerance).
- **End-to-end training:** Training on a synthetic dataset reduces the loss monotonically over epochs.
- **Packaging:** The project builds a wheel and can be installed via `pip install ..`

No quantitative performance claims are made; the tests merely assert that the logical relationships expected from the BT formulation hold.

5 Validation and Limitations

The test suite demonstrates that the implementation correctly encodes the Bradley–Terry probability model, that the synthetic preference generator respects the engineered comparator, and that the optimisation loop reduces the defined loss. These qualitative checks provide confidence that the code behaves as intended on synthetic data.

However, several aspects remain unevaluated:

- **Real-world trace quality:** The current comparator is hand-crafted; its alignment with true operational objectives is not validated.
- **Scalability:** The package has been exercised on modest trace pools (hundreds of examples). Large-scale orchestration scenarios may expose performance bottlenecks.
- **Generalisation:** No experiments assess whether a model trained on synthetic data transfers to traces generated by a different orchestrator or on real workloads.
- **Comparison to alternatives:** We have not benchmarked against other ranking or reward-learning methods.

These limitations define the scope of future empirical work.

6 Conclusion and Future Work

We introduced `orchestrator_rm`, a self-contained Python package that implements a Bradley–Terry reward model for multi-agent orchestration quality. The design emphasizes reproducibility, clear module boundaries, and a test suite that validates the core mathematical properties of the model. While the current implementation provides a solid foundation, empirical evaluation on realistic orchestration workloads, exploration of richer feature representations, and integration with reinforcement-learning controllers are natural next steps.

Acknowledgments

The author thanks the members of the Automate Capture Research group for feedback on the software design.